

⌕ CORE ENGINEERING HANDBOOK

Python & Software Engineering Research

An interview-ready technical breakdown of 28 foundational topics—spanning Python core syntax, OOP, concurrency, data stacks, SDLC methodologies, and automated tooling.

🐍 Python 3.12+

🔧 Software Architecture

🔍 Quality & Tooling

01 & 02. Indentation & Comments

Indentation Standard: Python uses leading spaces (4 spaces per block level) to define code block scope instead of curly braces {}.

Syntax Safety: Mixing tabs and spaces triggers an `IndentationError`. Visual clarity is enforced by compiler design.

Purposeful Comments: Lines starting with `#` explain *why* a decision was made, business logic exceptions, or constraints—not obvious code mechanics.

```
if score ≥ 40: # Business rule: Pass threshold status =  
"Pass" else: status = "Fail"
```

Interview Ready: Indentation defines scope and execution hierarchy. Comments document intent and non-obvious context, keeping code readable.



03 & 04. Functions & OOP Foundations

FUNCTIONAL REUSABILITY

3. Modular Functions

Functions encapsulate single logical tasks to eliminate duplication and enhance testability.

Parameters pass inputs; return produces pure outputs.

Supports default values, *args, and **kwargs.

```
def calc_net(gross, tax=0.2): return gross * (1 - tax)
```

Quick Answer: Reusable code block executing one single task.

OOP PILLARS

4. Object-Oriented Paradigm

Models real-world entities through Classes (blueprints) and Objects (instances).

Encapsulation: Bundles data and methods together.

Inheritance & Polymorphism: Extends base behavior cleanly.

```
class Employee: def __init__(self, name): self.name = name
```

Quick Answer: Paradigm using classes & objects for encapsulated systems.

05, 06 & 23. File I/O, Exceptions & Logging



5. File Handling

Safe file streaming using context managers (with open()) ensuring auto-closure.

```
with open("data.csv") as f:  
    records = f.read()
```

Context managers handle cleanup automatically.



6. Exception Handling

Prevents crashes using try, except, else, and finally logic blocks.

```
try: val = int(data) except  
ValueError: val = 0
```

Catch specific errors; avoid bare except.



23. Logging & Auditing

Records operational events across 5 standard severity levels (DEBUG to CRITICAL).

```
import logging  
logging.info("App booted")
```

Never log passwords or sensitive API tokens.

07 & 08. NumPy & Pandas Data Engineering

NUMERICAL COMPUTING

7. NumPy (Numerical Python)

Powers vectorized linear algebra using fast C-backed ndarray objects that outperform native Python lists.

```
import numpy as np marks = np.array([70, 80, 90]) mean_val = np.mean(marks) # 80.0
```

TABULAR ANALYSIS

8. Pandas DataFrames

High-level tool for data manipulation, filtering, aggregation, merging, and cleaning structured datasets.

```
import pandas as pd
df = pd.read_csv("emp.csv")
high_pay = df[df['salary'] > 50000]
```

coursera

Pandas Library Cheat Sheet

Importing Data

Actions	Description	Example Snippet
Import	Standard import statement to bring Pandas into the script.	<code>import pandas as pd</code>
Read_CSV	Reads a comma-separated values (CSV) file into DataFrame.	<code>df = pd.read_csv('file.csv')</code>
Read_Table	Reads a general delimited file into DataFrame.	<code>df = pd.read_table('file.txt')</code>
Read_Sql	Reads an SQL file into DataFrame.	<code>df = pd.read_sql('sql query')</code>
Read_SQL	Reads SQL query or database table into DataFrame.	<code>df = pd.read_sql('SELECT * FROM table', conn)</code>
Read_JSON	Reads a JSON formatted string into DataFrame.	<code>df = pd.read_json('file.json')</code>
Read_HTML	Reads HTML table into DataFrame.	<code>df = pd.read_html(url)</code>
Clipboard	Reads text from the clipboard into DataFrame.	<code>df = pd.read_clipboard()</code>

Create Test Objects

Action	Description	Example Snippet
Dataframe	Constructs a Dataframe object.	<code>df = pd.DataFrame(data)</code>
Series	Constructs a Series object.	<code>s = pd.Series(data)</code>
Index	Constructs an Index object.	<code>index = pd.Index(data)</code>

DataFrame Basics

Actions	Description	Example Snippet
Return dimensions of a DataFrame	Gets shape of DataFrame.	<code>df.shape</code>
Read CSV file into a DataFrame	Reads CSV and returns DataFrame object.	<code>df = pd.read_csv('File.csv')</code>
Return the data type of each column	Returns data type of columns in DataFrame.	<code>df.dtypes</code>

Selecting DataFrame Values

Actions	Description	Example Snippet
Select the rank column from (500)	Selects a specific column from Dataframe.	<code>TSDB["rank"]</code>
Select the first 3 rows from (500)	Shows the Dataframe.	<code>TSDB.head(3)</code>

ILOC / LOC

Actions	Description	Example Snippet
LDC	Access a group of rows and columns by labels.	<code>df.loc[row_labels, 'column_name']</code>
ILDC	Access a group of rows and columns by integer index.	<code>df.iloc[row_index, col_index]</code>

09 & 10. Web Scraping & Regex Engine



9. Selenium Automation

Automates real browser interaction to extract dynamic JavaScript-rendered web DOM elements.

Controls Chrome/Firefox via WebDriver API.

Supports Explicit Waits for asynchronous rendering.

Use official REST APIs whenever available over scraping.

Quick Answer: Browser automation tool for dynamic Web UI scraping.



10. Regular Expressions (Regex)

Pattern-matching syntax executed via the standard re module to validate and extract structured text.

```
import re email_pattern = r"[\w.-]+@[\w.-]+\.\w+"
match = re.search(email_pattern, text)
```

\d (digits), \w (words), + (1+), * (0+).

Quick Answer: Micro-language for text searching and pattern extraction.

11-13. Multithreading & Multiprocessing

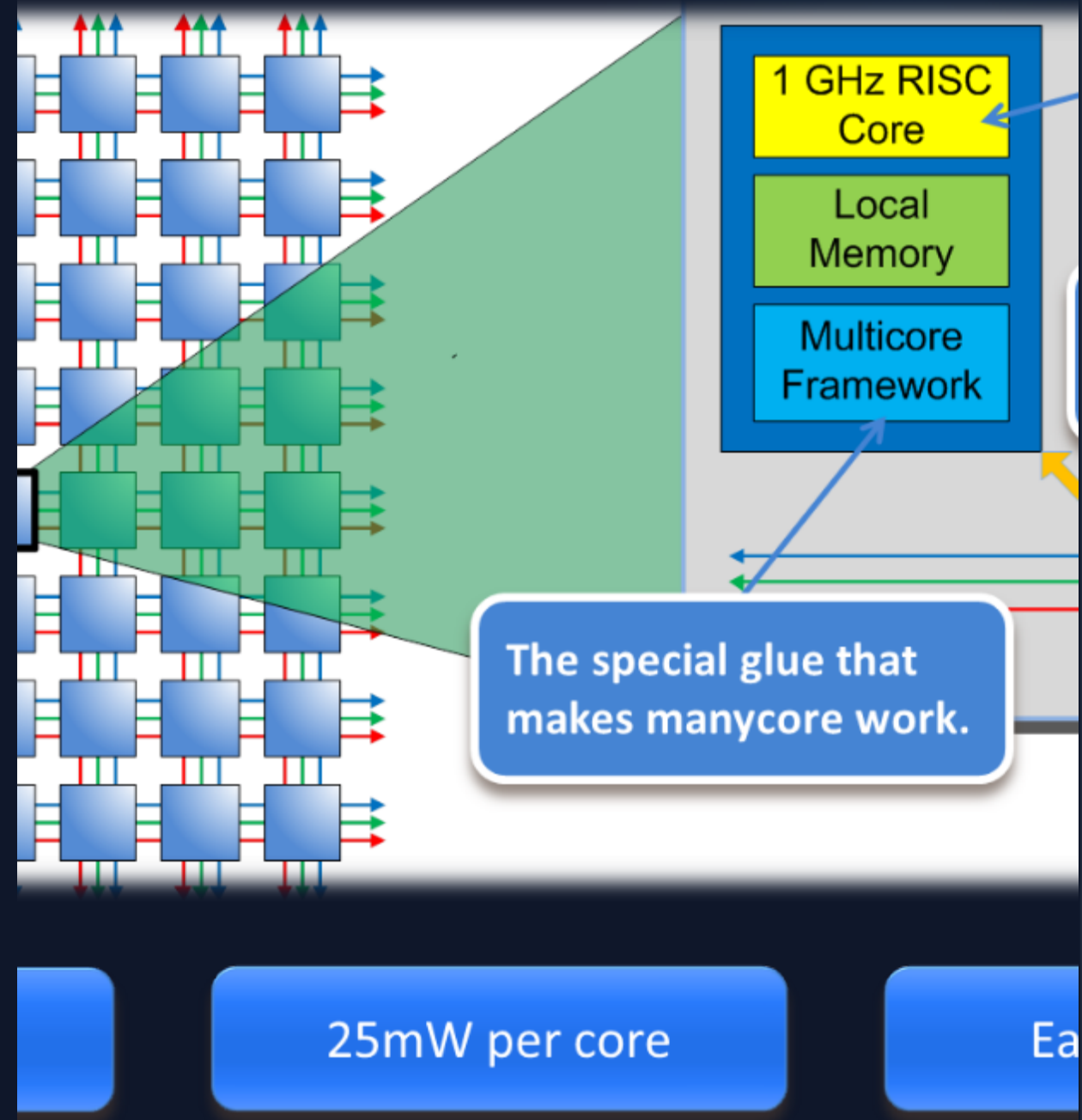
EXECUTION MODELS

Multithreading (I/O Bound): Multiple threads within one process share memory. Ideal for API calls & network waits. CPython's GIL prevents true parallel CPU execution.

Multiprocessing (CPU Bound): Spawns separate processes with independent memory spaces across multi-core CPUs.

Concurrency vs Parallelism: Concurrency is *dealing with* multiple tasks at once; Parallelism is *doing* multiple tasks simultaneously.

Memory Trick: Threading = 1 cook switching tasks. Parallelism = 3 cooks working at once.



14-16. SDLC, Agile Methodology & Scrum

STEP 01

14. SDLC Lifecycle

Structured phases: Requirements gathering, Design architecture, Coding, Testing, Deployment, and System Maintenance.

WATERFALL / ITERATIVE

STEP 02

15. Agile Philosophy

Iterative mindset focused on frequent delivery, feedback loops, customer collaboration, and rapid adaptiveness.

INCREMENTAL DELIVERY

STEP 03

16. Scrum Framework

Implements Agile using 2-4 week Sprints, Backlogs, Daily Standups, Sprint Reviews, and Retrospectives.

ROLES: PO, SM, DEVS

17-19. Git Workflows, Docs & Risk Ops

17. GIT VERSION CONTROL

Distributed system tracking code changes, enabling feature branching, merging, and Pull Request code reviews.

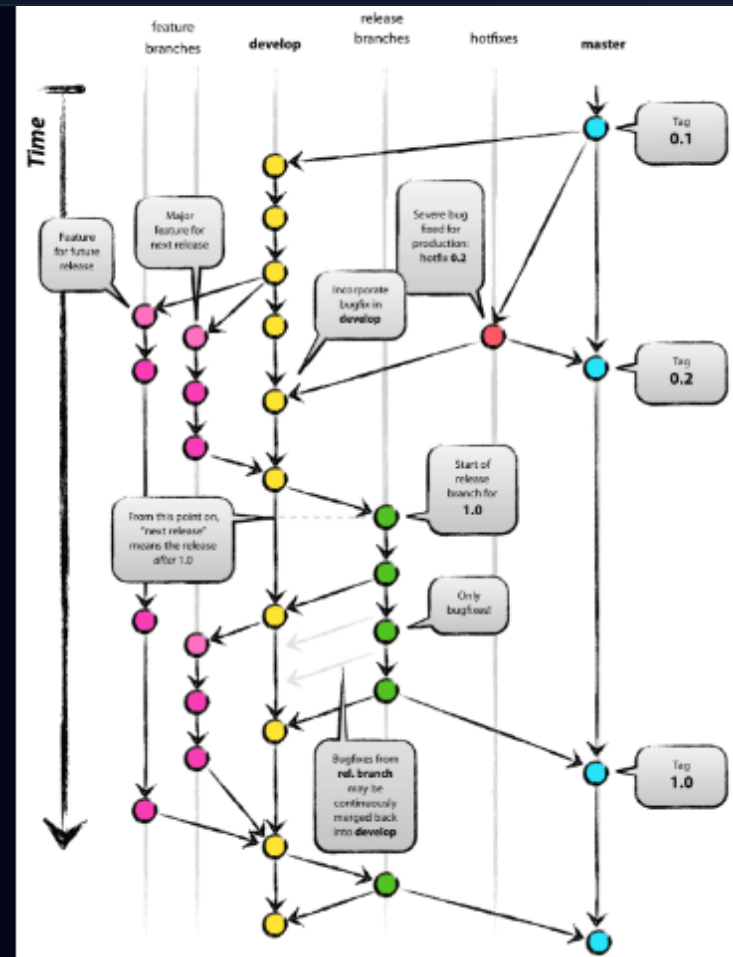
```
git checkout -b feature-login git commit -m "Add auth" git push origin feature-login
```

18. DOCUMENTATION STANDARD

Comprehensive README.md covering installation steps, config, environment setup, and API interfaces.

19. RISK MANAGEMENT MATRIX

Proactive assessment of potential failures (probability vs impact) with planned mitigations (e.g., automated backups).



20-22. Python Coding Standards & PEP 8

TEAM AGREEMENT

20. Coding Standards

Shared rules for consistent naming, file hierarchy, and error management across enterprise development teams.

Ensures uniform readability across devs.

STYLE GUIDE

21. PEP 8 Protocol

Official Python conventions: 4-space indent, 88-char line limits, snake_case functions, PascalCase classes.

Official Python style specification.

API INTERFACES

22. Docstrings vs Comments

Docstrings ("""Doc""") document public module/class/function APIs. Comments explain local inline logic.

Docstrings are inspectable via `help()`.

24 & 25. Efficiency & Architecture Principles

Principle / Concept	Definition & Purpose	Engineering Application
24. Efficient Code	Optimizing CPU, memory, database queries, and I/O resource consumption.	Use Hash Sets for $O(1)$ lookups instead of $O(n)$ lists. Profile before optimizing.
DRY (Don't Repeat Yourself)	Eliminate redundant logic by abstracting code into functions and modules.	Centralize shared business logic to prevent divergence and duplication bugs.
KISS & YAGNI	"Keep It Simple" and "You Aren't Gonna Need It". Avoid over-engineering.	Build exactly what is required now without premature speculative abstractions.
SOLID Principles	Five foundational OOP design guidelines (SRP, Open/Closed, Liskov, etc.).	Creates decoupled, modular, testable, and maintainable enterprise software.

26-28. Testing, Ruff Linter & Black



26. Unit Testing

Verifies isolated components produce expected outputs using unittest or pytest assertions in CI pipelines.

```
def test_add(): assert add(2,  
3) == 5
```

Validates runtime logic correctness.



27. Ruff Linter

An ultra-fast Rust-based static analyzer replacing Flake8 & isort. Detects bugs, dead imports, and style flaws in milliseconds.

```
ruff check --fix .
```

Lints and checks code quality instantly.



28. Black Formatter

An uncompromising, opinionated auto-formatter that standardizes formatting across teams to end PR layout debates.

```
black .
```

Formats code layout deterministically.